Snakefile implementation: comparison notes

This fork: KyryloFilonenko/reana-demo-bsm-search, commit 5b8d702
Upstream: reanahub/reana-demo-bsm-search, commit f94c1b1

Both trace back to the same Yadage analysis (`workflow/databkgmc.yml`) and were ported to Snakemake independently of each other. What follows is a side-by-side look at the two ports: rule structure, error handling, configuration, and whether the results are actually reproducible.

Reproducibility

This is the one area where my implementation is ahead, and it's the one that matters most for a physics analysis.

My `Snakefile` derives a per-job seed:

```
BASE_SEED = 42
SEED_OFFSET = {"data": 0, "sig": 100, "mc1": 200, "mc2": 300}
SHAPE_SEED_OFFSET = {"shape_conv_up": 1000, "shape_conv_dn": 2000}
```

`rule generate` computes `seed = BASE_SEED + SEED_OFFSET[sample] + batch` and passes it to `code/generantuple.py` , which calls `random.seed(seed)` before generating events. `rule mc_select_shape` does the same for the shape-systematic draw in `code/select.py` (`apply_shape()` calls `random.uniform()` per event, and without a seed that's a fresh number every run). I checked this empirically: two runs on lxplus produced byte-identical `postfit.pdf` files.

Upstream's `code/generantuple.py` is untouched from the original: no fourth argument, no `random.seed()` call anywhere. `rule generate` only passes `gentype` and `nevents` :

```
shell:
    shell_with_directories("""
    (
        python {input.script:q} {params.gentype:q} {params.nevents} {output:q}
    ) 2>&1 | tee {log:q}
    """)
```

Same story in `code/select.py` , the shape-variation draw is still unseeded. Every run of the upstream workflow produces a different `postfit.pdf` , a different histogram shape, and potentially a different fit result. For a demo workflow that's a tolerable gap, but it undercuts the basic premise that a REANA workflow should be reproducible.

	This fork	Upstream
Seeded event generation	yes	no

Seeded shape systematic	yes	no
Verified with repeat runs	yes	not applicable

Error visibility

My rules rely entirely on whatever `reana-client logs` decides to show. That turned out not to be enough: when `rule generate` was failing on `thisroot.sh: DYLD_LIBRARY_PATH: unbound variable`, REANA reported `Status: running` and `Step generate emitted no logs` for over half an hour, with nothing pointing at the actual cause. I only found the real error by reproducing the same run directly on lxplus, outside of REANA, where stderr was actually visible.

Upstream attaches a `log:` file to every single rule and tees stdout/stderr into it regardless of what REANA's own log viewer shows:

```
rule generate:
    ...
    log:
        "logs/generate/{sample}/{chunk}.log",
    shell:
        shell_with_directories("""
        (
            python {input.script:q} {params.gentype:q} {params.nevents} {output:q}
        ) 2>&1 | tee {log:q}
        """)
```

The identical `thisroot.sh` bug would have shown up immediately in `logs/generate/.../....log`, no half-hour wait and no need to reproduce it on a second machine. This is the single most expensive gap in my version, measured in the time it cost to diagnose things. Worth porting on its own regardless of anything else in this document.

Configuration

Mine is a plain Python dict inside the `Snakefile`:

```
PROFILES = {
    "full": {"batches": {"data": 5, ...}, "nevents": {"data": 20000, ...}, ...},
    "test": {"batches": {"data": 10, ...}, "nevents": {"data": 10000, ...}, ...},
}
```

No schema, no validation. A typo in a key surfaces only when the rule that reads it actually runs.

Upstream moved configuration into `workflow/config.yaml` and validates it against `workflow/config.schema.yaml` (JSON Schema, `additionalProperties: false`, explicit `required` fields) before Snakemake builds the DAG. A bad key or wrong type gets caught at parse time, not

after some jobs have already started. Their data model also stores `nevents` as an explicit per-chunk list rather than "batch count + events per batch," which allows uneven chunk sizes without touching code.

Shell argument safety

Upstream quotes every path substitution with Snakemake's built-in `{path:q}`:

```
"python {input.script:q} {input.data:q} {output:q} {wildcards.region:q} ..."
```

Mine substitutes paths directly:

```
"python code/select.py {input.root} {output} {wildcards.region} nominal"
```

With today's fixed, hardcoded filenames this is a latent issue rather than an active one. The moment a wildcard or config value accepts an arbitrary string (say, an output directory name pulled from config), unquoted substitution turns into a real command-injection risk.

Intermediate file cleanup

Upstream marks every intermediate output as `temp(...)`:

```
output:
    temp(f"{WORKDIR}/generated/{{sample}}/{{chunk}}.root"),
```

Snakemake deletes each one as soon as its last consumer in the DAG finishes. My version marks nothing as `temp`, so every intermediate `.root` file under `generated/`, `merged/`, `selected/`, `hists/`, `branch/` stays in the workspace indefinitely. That accumulation, combined with the container image cache, is part of why I ran into disk quota limits on lxplus and had to iterate through several storage locations before finding one that worked. `temp()` wouldn't have solved the image-cache problem specifically, but it would have kept the workspace itself a lot smaller.

Repository structure and tooling

	This fork	Upstream
Workflow file location	`Snakefile` at repo root	`workflow/Snakefile`
Configuration	inline in the Snakefile	`workflow/config.yaml` + schema
Dev environment manager	none	`pixi.toml` (`snakemake>=9,<10`, `graphviz`)

Dev tasks	none	`pixi run snakemake-dry-run` / `snakemake-lint` / `workflow-dag`
Tests	none	`tests/*.feature` (behave: log messages, run duration, workspace files)
Known-issues documentation	none	`docs/reana-snakemake-validation.md`

That last row is worth calling out on its own. Upstream documented a real incompatibility between a prerelease `reana-client`, Snakemake 9's config validation, and an outdated `jsonschema` pin, complete with reproduction steps and a link to the upstream issue. My fork has no equivalent write-up: the same class of knowledge (Docker TLS failures, AFS/EOS quota limits, the `thisroot.sh` unbound-variable bug) currently exists only as chat history, not as a searchable, version-controlled document.

Where we landed on the same fix independently

Two problems got solved the same way on both sides, which is a reasonable signal that both fixes are correct rather than accidental.

Directory creation. REANA doesn't reliably create a rule's output directory before the job container starts. Both ports compensate by running `mkdir -p` inside the rule's own shell command (`shell_cmd()` here, `shell_with_directories()` upstream).

thisroot.sh under strict mode. Snakemake runs every `shell:` block under `set -euo pipefail`. The legacy `thisroot.sh` script bundled with the ROOT 6.18 image reads `$DYLD_LIBRARY_PATH` without a safe default, and under `set -u` that's a fatal "unbound variable" error before any real work happens. Both ports turn nounset off locally around the `source` call:

```
# this fork
ROOTENV = "set +u && source /usr/local/bin/thisroot.sh && "
```

```
# upstream
"set +u\n"
"source /usr/local/bin/thisroot.sh\n"
"set -u\n"
```

Upstream's version is tidier. It turns `set -u` back on right after sourcing, while mine leaves nounset off for the rest of the rule's script.

Code from the shared workspace, not from the image. Both Dockerfiles are identical (`ADD code /code`, baking the code into the image at build time), and both ports resolved the same problem the same way: upload `code/` into the REANA workspace explicitly and call scripts by relative path (`code/x.py`) instead of the absolute in-image path (`/code/x.py`). The difference is

timing. Upstream built this in from their first Snakemake commit, while I only got there after discovering that the published image was silently running stale code.

Why upstream's Snakefile is longer

Upstream's `workflow/Snakefile` runs 561 lines across 22 rules. Mine is 359 lines across 18 rules, roughly 25.5 lines per rule there against 19.9 here. The gap maps directly onto the points above:

- **An extra merge step per branch.** Upstream keeps generated events split into per-batch chunks all the way through selection, then merges after selection with a dedicated rule (`merge_selected_data`, `merge_selected_signal`, and so on, four rules total). My `merge_generated` combines all batches into one file right after generation, so there's nothing left to merge after selection.
- **Per-rule logging.** The `log:` directive plus the `(...) 2>&1 | tee {log:q}` wrapper adds three lines to essentially every rule, about 60 lines across the file, all of it the exact thing that would have saved me the lxplus detour.
- **Explicit multi-file outputs.** `rule hepdata` upstream declares `zip=`, `submission=`, and `data=` as three tracked outputs; mine declares only the zip and leaves the other two as untracked side effects.
- **Config-driven helper functions.** Because nevents are stored as explicit per-chunk lists, upstream needs about seven small lookup functions (`generated_batches`, `nevents_for_chunk`, `selected_batch_files`, and so on) to translate config into wildcards. My flatter "batch count times events per batch" model needs none of that; `range(BATCHES[sample])` inline is enough.
- **A longer shared docstring.** The `shell_with_directories()` helper carries an eighteen-line explanation of both fixes and why they're temporary; my equivalent `shell_cmd()` docstring is about a third of that.

The length of the upstream file goes almost entirely into things already flagged above as its strengths: logging, explicit file tracking, more flexible configuration. It reflects a higher bar for engineering rigor, not padding.

Summary

Aspect	Ahead
Reproducibility	this fork
Error visibility / logging	upstream
Configuration validation	upstream
Shell argument safety	upstream
Intermediate file cleanup	upstream

Long-term maintainability	upstream
Readability for a newcomer	this fork
Tooling / version pinning	upstream
Known-issue documentation	upstream
Test coverage	upstream
REANA directory-creation workaround	tie
thisroot.sh nounset fix	tie
Code from workspace, not image	tie

Upstream wins 8 of 13. I'm ahead on 1, reproducibility. 3 are a wash.

Functionally, the two pipelines do exactly the same job: same inputs, same physics, same final plots and HEPData export. The difference sits entirely in the engineering built around that core. Upstream is the more robust implementation overall, with reproducibility as the one gap in an otherwise more disciplined design.